<u>Experiment-2</u>

Student Name: Pranav Chuahan
Branch: BE CSE (cyber security)
Semester: 5th
Subject Name: ADBMS

UID: 24BCY70004
Section/Group: 24AIT_KRG-G2
Date of Performance: 13/07/2026
Subject Code:24CSP-333

1. **Aim of the practical:** To learn and implement SQL queries in PostgreSQL using joins, grouping, aggregate functions, and sorting to generate analytical reports from relational database tables.
2. **Questions**

   (A) A university administration requires an analytical performance report for each academic unit. Write a PostgreSQL solution to calculate the average marks scored by students grouped by their respective departments. The solution must join the student profile, course subjects, and grade marks tables, returning a breakdown sorted by the highest average marks down to the lowest.

```sql
CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    dept_id INT REFERENCES Departments(dept_id)
);

CREATE TABLE Subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(50) NOT NULL
);

CREATE TABLE Marks (
    student_id INT REFERENCES Students(student_id),
    subject_id INT REFERENCES Subjects(subject_id),
    marks_scored INT NOT NULL,
    PRIMARY KEY (student_id, subject_id)
);
```

```sql
INSERT INTO Departments (dept_id, dept_name) VALUES
(1, 'Computer Science'),
(2, 'Mathematics');

INSERT INTO Students (student_id, student_name, dept_id)VALUES
(101, 'Alex', 1),
(102, 'Bella', 1),
(103, 'Charlie', 2);

INSERT INTO Subjects (subject_id, subject_name) VALUES
(501, 'Data Structures'),
(502, 'Calculus');

INSERT INTO Marks (student_id, subject_id, marks_scored) VALUES
(101, 501, 85),
(101, 502, 90),
(102, 501, 75),
(103, 502, 95);
```

Given tables:

Departments

dept_id [PK] integer	dept_name character varying (50)
1	Computer Science
2	Mathematics

Subjects

subject_id [PK] integer	subject_name character varying (50)
501	Data Structures
502	Calculus

Students

student_id [PK] integer	student_name character varying (50)	dept_id integer
101	Alex	1
102	Bella	1
103	Charlie	2

Marks

student_id [PK] integer	subject_id [PK] integer	marks_scored integer
101	501	85
101	502	90
102	501	75
103	502	95

Solution:

```sql
SELECT
    D.dept_name AS Department,
    ROUND(AVG(M.marks_scored), 2) AS "Average Marks"
FROM Departments D
JOIN Students S
    ON D.dept_id = S.dept_id
JOIN Marks M
    ON S.student_id = M.student_id
JOIN Subjects SU
    ON M.subject_id = SU.subject_id
GROUP BY D.dept_name
ORDER BY AVG(M.marks_scored) DESC;
```

department character varying (50)	Average Marks numeric
Mathematics	95.00
Computer Science	83.33

(B) Two legacy HR systems (A and B) have separate records of employee salaries. These records may overlap. Management wants to merge these datasets and identify each unique employee (by EmpID) along with their lowest recorded salary across both systems.

Objective

1. Combine two tables A and B.
2. Return each EmpID with their lowest salary, and the corresponding Ename.

Given tables:

```
--Q2
CREATE TABLE A
(
    EmpID INT,
    Ename VARCHAR(50),
    Salary INT
);

CREATE TABLE B
(
    EmpID INT,
    Ename VARCHAR(50),
    Salary INT
);

INSERT INTO A (EmpID, Ename, Salary)
VALUES
(1, 'AA', 1000),
(2, 'BB', 300);

INSERT INTO B (EmpID, Ename, Salary)
VALUES
(2, 'BB', 400),
(3, 'CC', 100);
```

TABLE A

empid integer	ename character varying (50)	salary integer
1	AA	1000
2	BB	300

TABLE B

empid integer	ename character varying (50)	salary integer
2	BB	400
3	CC	100

Solution:

```
select
    EmpID , min(Ename) AS ENAME,
    min(Salary) AS SALARY
from
(
select * from A
union all
select * from B
) as C
GROUP BY EMPID
order by EMPID ;
```

empid integer	ename text	salary integer
1	AA	1000
2	BB	300
3	CC	100

(C) A multinational organization maintains separate records for employee information and department details. The management wants to identify the employees who receive the highest salary within their respective departments for an annual performance report.

1) If more than one employee earns the same highest salary in a department, all such employees should be included in the report. The final output should display the Department Name, Employee Name, and Salary, arranged in ascending order of department name.
   Note: The employee and department information are maintained in separate normalized tables.

Given tables:

```
----q3
CREATE TABLE Department
(
    id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

CREATE TABLE Employee
(
    id INT PRIMARY KEY,
    name VARCHAR(50),
    salary INT,
    department_id INT,
    FOREIGN KEY(department_id)
    REFERENCES Department(id)
);
```

```
INSERT INTO Department
VALUES
(1,'IT'),
(2,'SALES');

INSERT INTO Employee
VALUES
(1,'JOE',70000,1),
(2,'JIM',90000,1),
(3,'HENRY',80000,2),
(4,'SAM',60000,2),
(5,'MAX',90000,1);
```

Department

id [PK] integer	dept_name character varying (50)
1	IT
2	SALES

Employee

id [PK] integer	name character varying (50)	salary integer	department_id integer
1	JOE	70000	1
2	JIM	90000	1
3	HENRY	80000	2
4	SAM	60000	2
5	MAX	90000	1

Solution:

```
select
    d.dept_name,
    e.name,
    e.salary
from
Employee e
join
Department d
on e.department_id=d.id
where e.salary in
(
    select max(e1.salary)
    from Employee e1
    where e1.department_id=e.department_id
);
```

dept_name character var	name character	salary integer
IT	JIM	90000
SALES	HENRY	80000
IT	MAX	90000

2) Find the 2ND highest earning EMPLOYEE from the same schema

```
SELECT
    d.dept_name,
    e.name,
    e.salary
FROM Employee e
JOIN Department d
ON e.department_id = d.id
WHERE e.salary = (
    SELECT MAX(e1.salary)
    FROM Employee e1
    WHERE e1.department_id = e.department_id
      AND e1.salary < (
          SELECT MAX(e2.salary)
          FROM Employee e2
          WHERE e2.department_id = e.department_id))
ORDER BY d.dept_name;
```

dept_name character var	name character	salary integer
IT	JOE	70000
SALES	SAM	60000

3. Learning Outcomes: Install and configure MySQL Server and MySQL Workbench.

- Learned to create and maintain relational database tables using PostgreSQL.
- Gained hands-on experience with SQL JOINs to retrieve data from multiple tables.
- Applied aggregate functions (AVG, MAX, MIN) for data analysis and reporting.
- Developed SQL queries using subqueries, GROUP BY, and ORDER BY clauses.
- Improved problem-solving skills by implementing real-world database query scenarios.

Evaluation Grid:

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Student Performance (Conduct of experiment)		12
2.	Viva Voce		10
3.	Submission of Work Sheet (Record)		8
	Signature of Faculty (with Date):	Total Marks Obtained:	30